

Lecture 16

Immutable Data Types

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Elm Resources

- References
 - <https://guide.elm-lang.org/>
 - <https://elmprogramming.com/>
 - <https://elm-lang.org/examples>
- Online editors
 - <https://ellie-app.com/>
 - <https://elm-lang.org/try>
 - <https://elm-editor.com/>

Modules

Module: Collection of functions and types

- Similar to Python imports
- Added to Elm programs via import: `import List`
- Example: List module
 - Contains functions for manipulating lists
 - No need to import; imported by default

Modules

- Different ways of importing modules:

```
-- Import just ModuleName  
import ModuleName
```

```
-- Import ModuleName, ModuleName.Type1, ModuleName.func1, and  
ModuleName.func2  
import ModuleName exposing (Type1, func1, func2)
```

```
-- Import ModuleName and everything inside of it  
import ModuleName exposing (..)
```

Modules

- All Elm programs have the following imports by default (docs):

```
import Basics exposing (..)          -- Everything inside Basics
import List    exposing (List, (:::)) -- List type and (:::)
import Maybe   exposing (Maybe(..)) -- Maybe type and all its
variants
import String  exposing (String)      -- String type
import Char    exposing (Char)        -- Char type
import Tuple
import Debug
```

Lists

- `List a`: Standard container for sequences of values of type `a`
- Elements are of the same type (homogeneous)
- `a` is a type variable that refers to any type
- `List Int`: `List` of integers
- `List String`: `List` of strings
- `List (List Int)`: `List` of (list of integers)

Lists

- Creation:

```
x = []           -- Empty list of type `List a`  
  
y = [1]         -- List of type `List Int` with one element  
  
z = ["a", "b"]  -- List of type `List String` with two elements
```


List Module: Creation

`List.repeat : Int -> a -> List a`

- Arguments: n, value
- Returns a list with n instances of value

```
List.repeat 3 "hi" -- ["hi","hi","hi"]
```

```
List.repeat 5 0 -- [0,0,0,0,0]
```

```
List.repeat 3 [0] -- [[0],[0],[0]]
```

List Module: Creation

Self-check: `zeroMatrix: Int -> Int -> List (List Int)`

- rows, cols
- All inner elements must be 0

List Module: Creation

`List.range : Int -> Int -> List Int`

- Arguments: `start`, `end`
- Returns a list of integers from interval `[start, end]`

```
List.range 1 5 -- [1,2,3,4,5]
```

```
List.range 5 1 -- []
```

```
getIndices : Int -> List Int
```

```
getIndices n = -- getIndices 3 == [0,1,2]  
    List.range 0 (n - 1)
```

List Module: Creation

- Self-check: Make a function `f` for the output below:
- 3 rows, 4 columns:

```
[[1, 2, 3, 4],  
 [1, 2, 3, 4],  
 [1, 2, 3, 4]]
```

- What about for `r` rows, `c` columns?

List Module: Creation

`(::)` : $a \rightarrow \text{List } a \rightarrow \text{List } a$

- Pronounced “cons” (imported by default)
- Arguments: `value`, `lst`
- Prepends `value` to `lst`

List Module: Creation

```
(::) 1 [2, 3]      -- [1,2,3]
```

- Can be called in infix style:

```
1 :: []           -- [1]

1 :: [2, 3]       -- [1,2,3]
1 :: (2 :: [3])   -- [1,2,3]
1 :: (2 :: (3 :: [])) -- [1,2,3]
1 :: 2 :: 3 :: [] -- [1,2,3]
```

- A list can be viewed as a chain of conses

- “Linked list”; more on this next time

Maybe

`List.maximum [10, 100, 20, 30]`

- Correct answer is 100

`List.maximum [10]`

- Correct answer is 10

`List.maximum []`

- What about this?

Maybe

- **Maybe** a: Wrapper type representing presence or absence of a value of type a
- Also called Option or Optional elsewhere
- Exactly one of two possibilities:
 - **Just** value: Present (value is of type a)
 - **Nothing**: Absent

Maybe

Syntax:

```
x = Just 10           -- Maybe Int
y = Just "hello"      -- y : Maybe String
z = Just [1, 2, 3]    -- z : Maybe (List Int)
w = Nothing           -- w : Maybe a
```

```
-- List.maximum : List Int -> Maybe Int
a = List.maximum [10,100,20,30] -- Just 100
b = List.maximum []             -- Nothing
```

Handling Maybe Values

- Option 1: Pattern matching via case:

```
unwrap : Maybe a -> String
unwrap box =
    case box of
        Just gift ->
            "The box contains " ++ Debug.toString gift

        Nothing ->
            "The box is empty"
```

- Must always have branches for both cases

- Forces you to deal with the unhappy path

Handling Maybe Values

- Option 2: `Maybe.withDefault` : `a -> Maybe a -> a`:

```
x =  
    -- String.toInt : String -> Maybe Int  
    String.toInt "-123"  
    |> Maybe.withDefault 0    -- x == -123
```

- Unboxes if exists, else provides default
- Useful when data is sure to be “clean”

List Module: Other functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
length	List a -> Int	Gives length of lst	List.length [11, 12]
reverse	List a -> List a	Reverses lst	List.reverse [11, 12]
member	a -> List a -> Bool	Determines existence of value in lst	List.member 10 [11, 12]

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
sum	List num -> num	Gives sum of 1st (default 0)	List.sum [11, 12]

List Module: Other functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
append	List a -> List a a -> List a	Combines lst1 and lst2	List.append [1, 10], [11, 12]
concat	List (List a) -> List a	Flattens list of lists into single list	List.concat [[1, 10], [11, 12]]

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
sort	<code>a -> List cmp -> List cmp</code>	Sorts list in ascending order	<code>List.sort [12, 10, 11]</code>

List Module: Other functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
take	<code>Int -> List a -> List a</code>	Gives first <code>n</code> members of <code>lst</code>	<code>List.take 2</code> <code>[10, 11, 12]</code>
drop	<code>Int -> List a -> List a</code>	Removes first <code>n</code> members of <code>lst</code>	<code>List.drop 2</code> <code>[10, 11, 12]</code>
maximum	<code>List cmp -> Maybe cmp</code>	Gives maximum of <code>lst</code> (if exists)	<code>List.maximum</code> <code>[12, 10, 11]</code>

List Module: Other functions

- Is there a function that gets the element at index i of the list?
 - Is `List` the best data type to do this?

Arrays

- `Array a`: Container for random-access processing of a list of values
- Elements are of the same type (homogeneous)
- Creation is usually done via a list with `fromList`:
 - `fromList : List a -> Array.Array a`

Arrays

```
import Array  -- Not imported by default

-- Empty array of type `Array a`
x = Array.fromList []

-- Array of type `Array Int` with one element
y = Array.fromList [1]

-- Array of type `Array String` with two elements
z = Array.fromList ["a", "b"]
```

Array Module vs. Array Type

`Array` module contains `Array` type

- Module is not imported by default
- To access Array type inside Array module: `Array.Array`

Array Module vs. Array Type

```
import Array
```

```
x : Array.Array Int      -- Has name of module
```

```
x = Array.fromList [1]
```

Better to import directly via exposing:

```
import Array exposing (Array)
```

```
x : Array Int           -- No name of module
```

```
x = Array.fromList [1]
```

Random-Access Array Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
get	<code>Int -> Array a -> Maybe a</code>	Gives element in index i of arr (if exists)	<code>get 0 (fromList ["a", "b"])</code>
set	<code>Int -> a -> Array a -> Array a</code>	Sets index i of arr if i is valid index	<code>set 0 "x" (fromList ["a", "b"])</code>

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
slice	<code>Int -> Int -> Array a -> Array a</code>	Gets subarray of arr from index start to end	<code>slice 1 2 (fromList ["a", "b", "c", "d"])</code>

Other Array Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
empty	<code>Array a</code>	Gives an empty array	<code>empty</code>
repeat	<code>Int -> a -> Array a</code>	Makes an array with n members of value	<code>repeat 2 "a"</code>
isEmpty	<code>Array a -> Bool</code>	Determines emptiness of arr	<code>isEmpty empty</code>

Other Array Functions

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
length	<code>Array -> Int</code>	Gives length of arr	<code>length (fromList [11, 12])</code>
push	<code>a -> Array a -> Array a</code>	Adds val to end of arr	<code>push "c" (fromList ["a", "b"])</code>
toList	<code>Array a -> List a</code>	Makes list from arr	<code>toList (fromList</code>

<i>Function</i>	<i>Signature</i>	<i>Description</i>	<i>Usage</i>
			[10, 11, 12])
toIndexedList	Array a -> List (Int, a)	Makes list of index- element pairs from arr	toIndexedList (fromList ["a", "b", "c"])

Immutability of Containers

- *Immutable*: Cannot be changed
 - Elm is a pure (fully immutable) language
 - Variables cannot be mutated
 - Values cannot be mutated
 - Containers cannot be mutated
- “Adding” elements to a container does not change the container
 - A new container is returned equivalent to the result

Immutability of Containers

```
x = Array.fromList [10, 20, 30]
```

```
y = Array.set 2 100 x
```

```
_ = Debug.log "" x -- Array.fromList [10,20,30]; did not change
```

```
_ = Debug.log "" y -- Array.fromList [10,20,100]; result is here
```

List Filter

`List.filter : (a -> Bool) -> List a -> List b`

- Takes in a predicate and a list
- *Predicate*: Function that returns a `Bool`

List Filter Examples

- `isOdd : Int -> Bool`

```
isOdd : Int -> Bool
```

```
isOdd n =
```

```
    modBy 2 n /= 0
```

```
x = List.range 1 10
```

```
y = List.filter isOdd x  -- [1,3,5,7,9]
```


List Filter Examples

- `isEven : Int -> Bool`

```
isEven : Int -> Bool
```

```
isEven n =
```

```
    isOdd n |> not
```

```
z = List.filter isEven x  -- [2,4,6,8,10]
```

Tuples

Tuple: Pair or triple of values

- Useful for returning groups of multiple values at once
 - Use lists or records for more than three (next time)
- May contain different types:

```
-- x : (Int, String, Bool)
x = (1, "a", True)

-- y : ((Int,String),(Int,String),(Int,String))
y = ((0,"x"),(1,"y"),(2,"z"))
```

Tuples

- Tuple destructuring via pattern matching (only in `let`):

```
x = (3, 4, 5)

answer =
  let
    (a, _, c) = x  -- Underscore for "don't care"
  in
    a^2 + c^2
```

Self-Check 1

- `countUpperLower : String → (Int, Int)`

```
countUpperLower "HeLL0 WorLD" == (7,3)
```

```
countUpperLower "DRACARYS" == (8,0)
```

```
countUpperLower "tyrion lannister" == (0,15)
```

```
countUpperLower "targarYEN" == (3,6)
```

Mapping Intuition

- Suppose we have a list of Ingredients:

```
[🐮, 🥚, 🐔] -- List Ingredient
```

- We want to 🔥 each Ingredient in the List separately:

```
🔥 🐮 -> 🥩 -- 🔥 : Ingredient -> Food
```

```
🔥 🥚 -> 🍳
```

```
🔥 🐔 -> 🍗
```

Mapping Intuition

- We can use `List.map` to 🔥 each ingredient:

```
List.map 🔥 [🐮, 🥚, 🐔]
```

```
-- Does the following:
```

```
-- [🐮, 🥚, 🐔]
```

```
-- ↓ ↓ ↓
```

```
-- ↓ ↓ ↓
```

```
-- 🔥 🔥 🔥
```

```
-- ↓ ↓ ↓
```

-- ↓ ↓ ↓

-- [🍖, 🔍, 🍗]

-- And produces [🍖, 🔍, 🍗]

List Mapping as Restricted “Iteration”

- `List.map` captures the common pattern of iterating through a list to replace each element using some block of logic (i.e., function)

increment example in Python vs. in Elm:

```
def increment(n):  
    return n + 1;           # increment(10) == 11  
  
nums = [10, 20, 30]
```



```
for i in range(len(nums)):
    nums[i] = increment(nums[i])
```

```
increment : Int -> Int
increment n =                -- increment 10 == 11
    n + 1

x = [10, 20, 30]

y = List.map increment x    -- [11,21,31]
```

List Mapping

- `List.map : (a -> b) -> List a -> List b`

```
isOdd : Int -> Bool          -- isOdd 1 == True
isOdd n = modBy 2 n == 0

parityStr : Int -> String    -- parityStr 1 == "odd"
parityStr n =
    if isOdd n then "odd" else "even"

-- Same as `List.map parityStr (List.range 1 5)`
x =
    List.range 1 5           -- [ 1 , 2 , 3 , 4 , 5 ]
```

```
|> List.map parityStr -- ["odd","even","odd","even","odd"]
```

List Mapping

- Important consequence: Number of elements stays the same
- `List.map` can neither add nor remove elements

Map in other programming languages

- Python:

```
def increment(n: int) -> int:  
    return n + 1
```

```
x = [10, 20, 30]
```

```
print(list(map(increment, x)))
```

```
# [11, 21, 31]
```

```
# `list()` is needed to  
"execute" the map
```

Map in other programming languages

- Compare map with Python's list comprehension:

```
x = [10, 20, 30]
print([increment(k) for k in x]) # Simple list comprehension as a
"map";
                                # same principle
```

Map in other programming languages

- JavaScript has `map` too:

```
const increment = n => n + 1;  
console.log([10, 20, 30].map(increment));
```

Self-Check 2

- `countDigitOccurrences` : `Int -> Int -> Int`

```
countDigitOccurrences 1000000 0 == 6
```

```
countDigitOccurrences 123 3 == 1
```

```
countDigitOccurrences 39284344 4 == 3
```

```
countDigitOccurrences 2883777 6 == 0
```

```
countDigitOccurrences 9182837 9 == 1
```


Self-Check 3

- `isPrime : Int -> Bool`

```
isPrime 2 == True
isPrime 3 == True
isPrime 4 == False
isPrime 5 == True
isPrime 6 == False
isPrime 7 == True
isPrime 8 == False
isPrime 9 == False
```

Self-Check 4

- `invertedTriangle : Int → String`

```
invertedTriangle 1
```

```
-- *
```

```
invertedTriangle 2
```

```
-- **
```

```
-- *
```

```
invertedTriangle 3
```

```
-- ***
```

```
-- **
```

```
-- *
```

```
invertedTriangle 4
```

```
-- ****
```

```
-- ***
```

```
-- **
```

```
-- *
```

Elm Collection Types

<i>Container</i>	<i>Purpose</i>
List	Sequential access of elements
Array	Random access of elements
String	Text manipulation
Set	Uniqueness and set operations
Dict	Associating pairs of values
Maybe	Possible absence of value

Elm Cheatsheet

https://docs.google.com/spreadsheets/d/e/2PACX-1vSgPYdEM3O04mHE-g2-uUCbLFNHU3p52S9ArhKQ-9ySbOWgHw7sLnjh3g_iCUFwqOJYf-SFXLb07Cl8/pubhtml